

# String Matching

## Definition:

Let  $T$  be a text of length  $n$  given in an array  $T[1 \cdots n]$ , and let  $P$  be a pattern of length  $m$  given in an array  $P[1 \cdots m]$ ,  $n > m$ .

We say that  $P$  appears in  $T$  with shift  $s$ , if  $0 \leq s \leq n - m$  and  $P[1 \cdots m] = T[s + 1 \cdots s + m]$ .

If  $P$  appears in  $T$  with shift  $s$ , we say that  $s$  is a **valid shift**, otherwise,  $s$  is an **invalid shift**.

In the **string matching problem** we want to find all the valid shifts of  $P$  in  $T$ .

# String Matching

Example:

T

|   |   |   |   |   |   |   |   |   |    |
|---|---|---|---|---|---|---|---|---|----|
| a | b | a | b | b | a | b | b | a | b  |
| 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 |

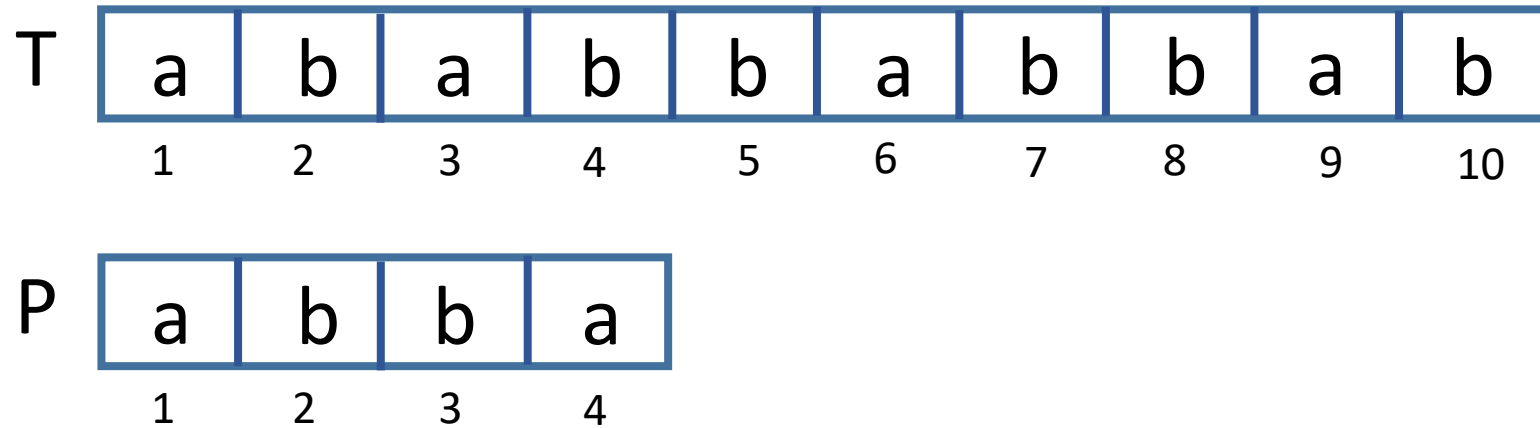
P

|   |   |   |   |
|---|---|---|---|
| a | b | b | a |
| 1 | 2 | 3 | 4 |

S= ?

# String Matching

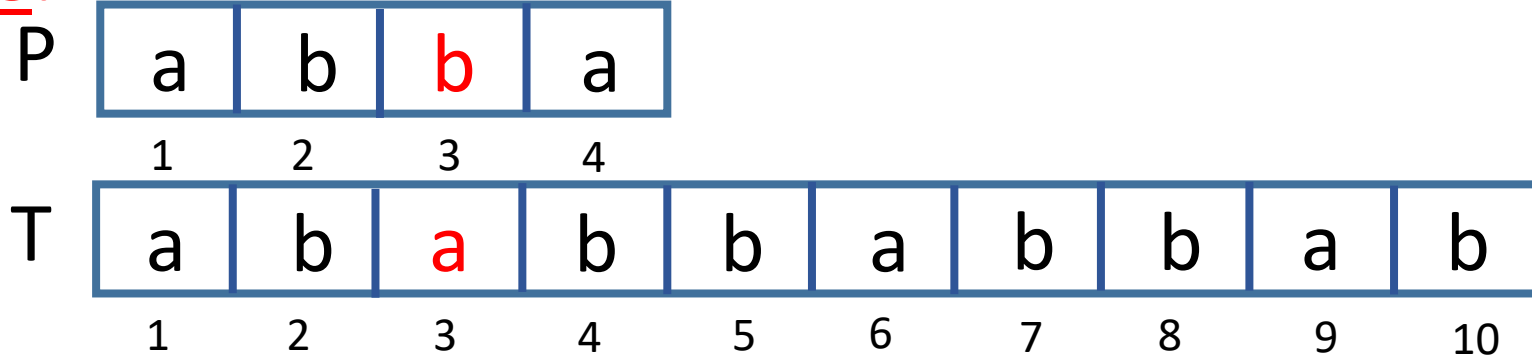
Example:



$$s = 0, 1, 2, 3, 4, 5, 6 = 10 - 4$$

# String Matching

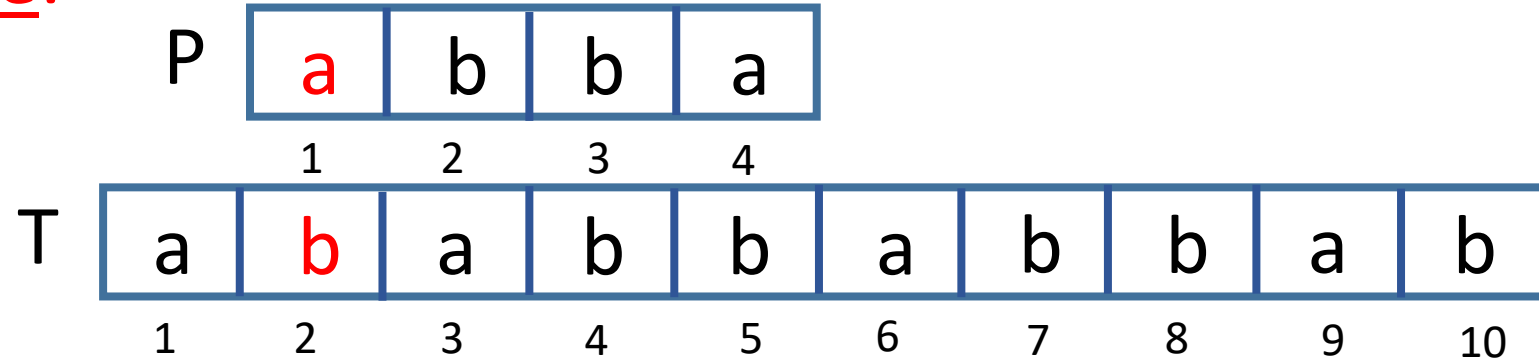
Example:



$$s = \cancel{0}, 1, 2, 3, 4, 5, 6 = 10 - 4$$

# String Matching

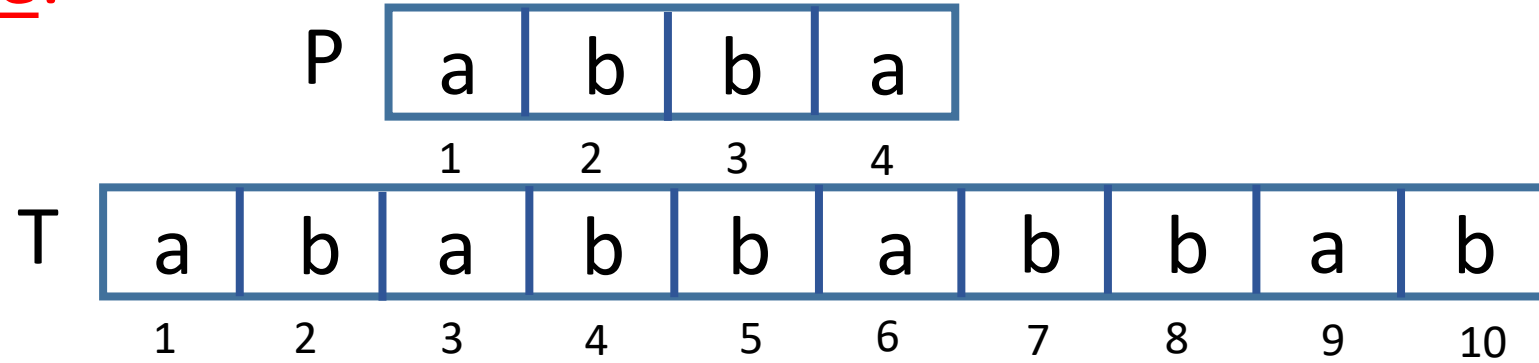
Example:



$$s = \cancel{0}, \cancel{1}, 2, 3, 4, 5, 6 = 10 - 4$$

# String Matching

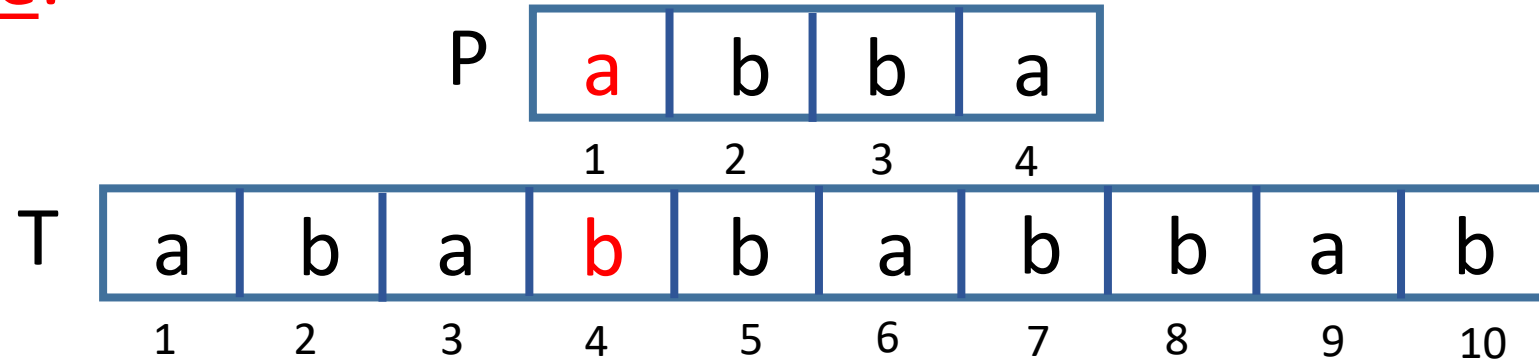
Example:



$$s = \cancel{0}, \cancel{1}, (2), 3, 4, 5, 6 = 10 - 4$$

# String Matching

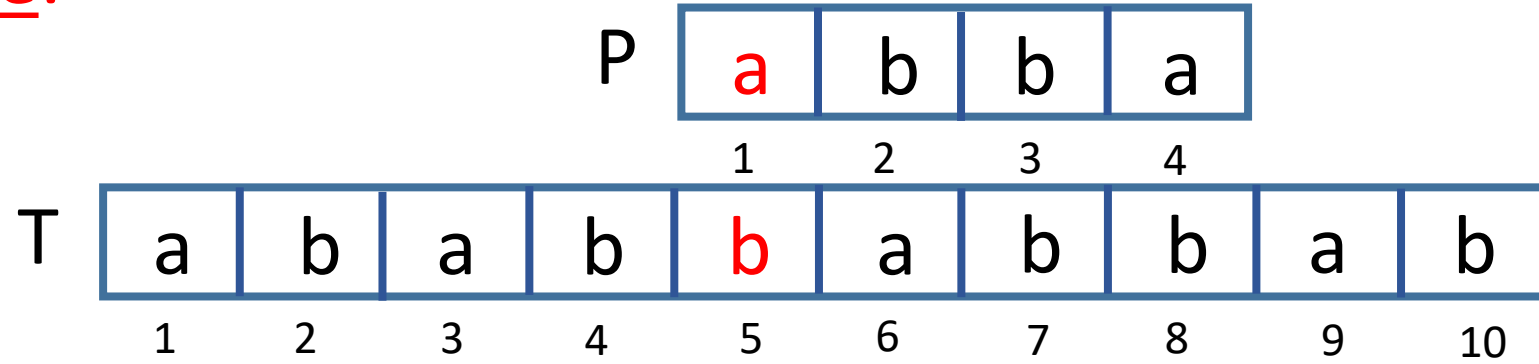
Example:



$$s = \cancel{0}, \cancel{1}, \textcircled{2}, \cancel{3}, 4, 5, 6 = 10 - 4$$

# String Matching

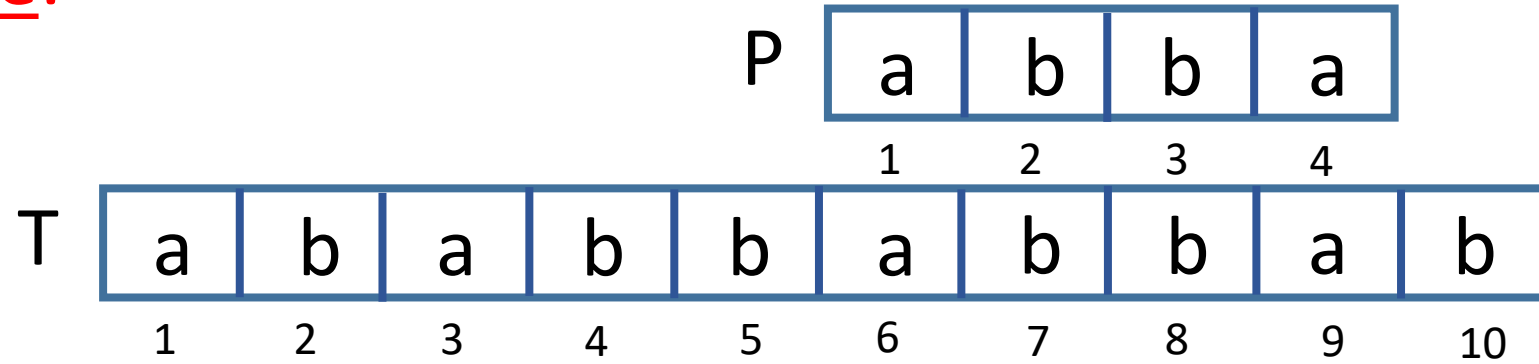
Example:



$$s = \cancel{0}, \cancel{1}, \textcircled{2}, \cancel{3}, \cancel{4}, 5, 6 = 10 - 4$$

# String Matching

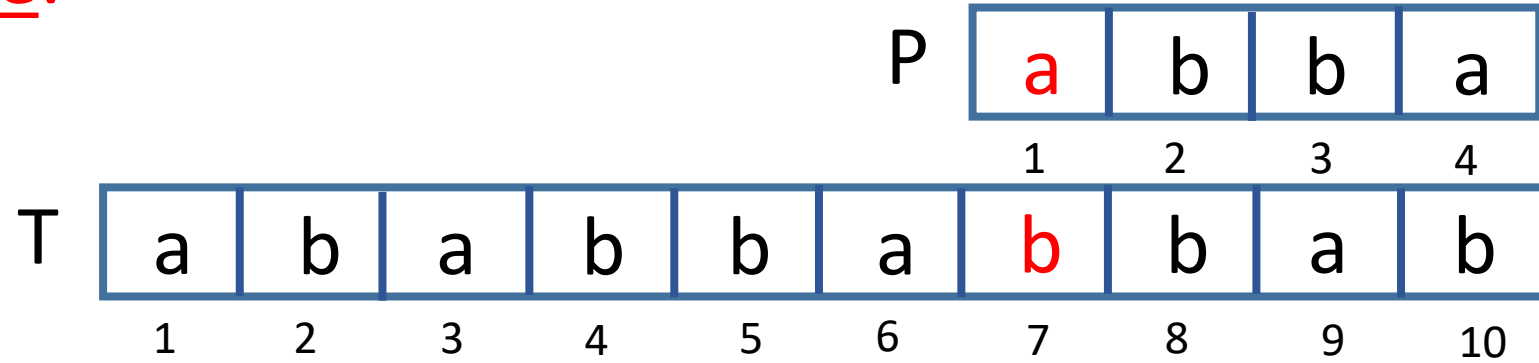
Example:



$$s = \cancel{0}, \cancel{1}, \textcircled{2}, \cancel{3}, \cancel{4}, \textcircled{5}, 6 = 10 - 4$$

# String Matching

Example:



$$s = \cancel{0}, \cancel{1}, \textcircled{2}, \cancel{3}, \cancel{4}, \textcircled{5}, \cancel{6} = 10 - 4$$

# String Matching – Naïve Algorithm

Example:

|   |   |   |   |   |   |   |   |   |   |    |
|---|---|---|---|---|---|---|---|---|---|----|
| T | a | b | a | b | b | a | b | b | a | b  |
|   | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 |

|   |   |   |   |   |
|---|---|---|---|---|
| P | a | b | b | a |
|   | 1 | 2 | 3 | 4 |

The cost of the naïve algorithm is:

$$O(n \cdot m)$$

$s = \cancel{0}, \cancel{1}, \textcircled{2}, \cancel{3}, \cancel{4}, \textcircled{5}, \cancel{6} = 10 - 4$

Is this bound tight?

# String Matching – Naïve Algorithm

Example:

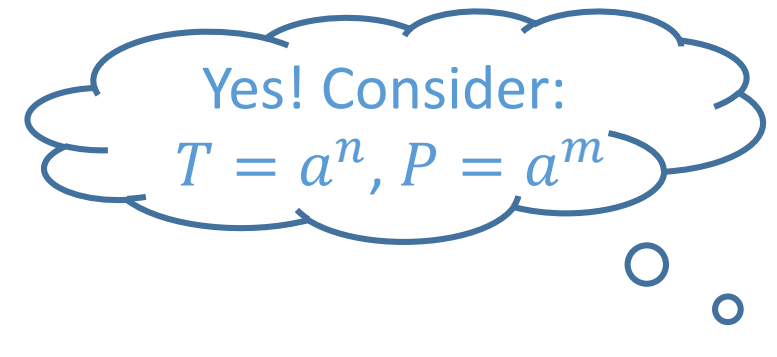
|   |   |   |   |   |   |   |   |   |   |    |
|---|---|---|---|---|---|---|---|---|---|----|
| T | a | b | a | b | b | a | b | b | a | b  |
|   | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 |

|   |   |   |   |   |
|---|---|---|---|---|
| P | a | b | b | a |
|   | 1 | 2 | 3 | 4 |

The cost of the naïve algorithm is:

$$O(n \cdot m)$$

$s = \cancel{0}, \cancel{1}, \textcircled{2}, \cancel{3}, \cancel{4}, \textcircled{5}, \cancel{6} = 10 - 4$



# String Matching – Rabin-Karp Algorithm

## Theorem: (Correctness of Rabin-Karp)

Let  $T$  be a text of length  $n$  given in an array  $T[1 \cdots n]$ , and let  $P$  be a pattern of length  $m$  given in an array  $P[1 \cdots m]$ ,  $n > m$ .

Then, Rabin-Karp algorithm reports all and only the valid shifts of  $P$  in  $T$ .

## Proof:

First note that every reported valid shift is indeed valid, since the only report occurs in line 12 which we reach only after  $m$ -subsequent matches are detected in line 11.

Also, any shift that is not reported is indeed invalid: either rejected with non-equal residue in line 10 or by a mismatch in line 11.

# String Matching – Rabin-Karp Algorithm

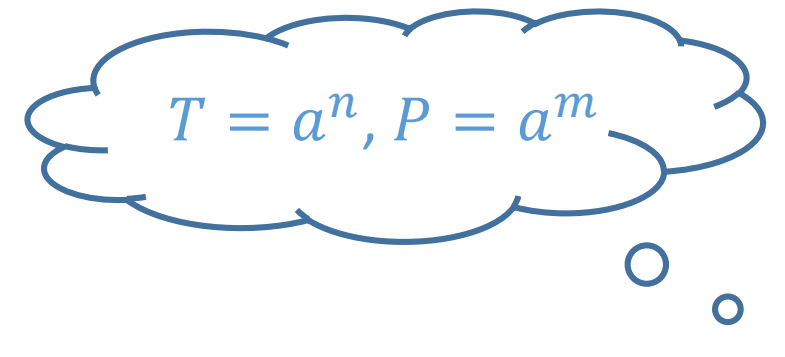
Example:

|   |   |   |   |   |   |   |   |   |   |    |
|---|---|---|---|---|---|---|---|---|---|----|
| T | a | b | a | b | b | a | b | b | a | b  |
|   | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 |

|   |   |   |   |   |
|---|---|---|---|---|
| P | a | b | b | a |
|   | 1 | 2 | 3 | 4 |

$s = \cancel{0}, \cancel{1}, \textcircled{2}, \cancel{3}, \cancel{4}, \textcircled{5}, \cancel{6} = 10 - 4$

The cost of Rabin-Karp algorithm is:  
 $O(n \cdot m)$



# String Matching – Rabin-Karp Algorithm

Example:

|   |   |   |   |   |   |   |   |   |   |    |
|---|---|---|---|---|---|---|---|---|---|----|
| T | a | b | a | b | b | a | b | b | a | b  |
|   | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 |

|   |   |   |   |   |
|---|---|---|---|---|
| P | a | b | b | a |
|   | 1 | 2 | 3 | 4 |

$s = \cancel{0}, \cancel{1}, \textcircled{2}, \cancel{3}, \cancel{4}, \textcircled{5}, \cancel{6} = 10 - 4$

The cost of Rabin-Karp algorithm is:

$$\Theta(n) + O\left(m \cdot \left(v + \frac{n}{q}\right)\right)$$

Number of  
valid shifts

Upper bound  
on invalid  
shifts with  
equal residue

# String Matching – Rabin-Karp Algorithm

Example:

|   |   |   |   |   |   |   |   |   |   |    |
|---|---|---|---|---|---|---|---|---|---|----|
| T | a | b | a | b | b | a | b | b | a | b  |
|   | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 |

|   |   |   |   |   |
|---|---|---|---|---|
| P | a | b | b | a |
|   | 1 | 2 | 3 | 4 |

$s = \cancel{0}, \cancel{1}, \textcircled{2}, \cancel{3}, \cancel{4}, \textcircled{5}, \cancel{6} = 10 - 4$

The cost of Rabin-Karp algorithm is:

$$\Theta(n) + O\left(m \cdot \left(v + \frac{n}{q}\right)\right) \\ = O(n + m)$$

If  $v$  is  $O(1)$

If  $q > m$  then  
 $\frac{m}{q} < 1$ , so  
 $m \cdot \frac{n}{q} = O(n)$

# String Matching

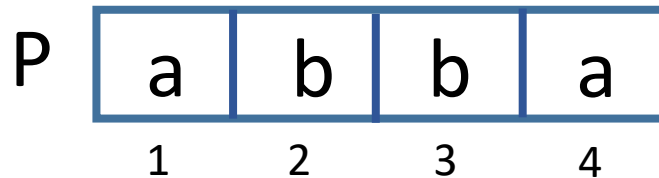
## Definitions:

Let  $S$  be a string of length  $n$  given in an array  $S[1 \cdots n]$ .

We say that  $P_k$  is a **prefix** of  $S$  with length  $k$  if  $P_k = S[1 \cdots k]$ .

We say that  $S_k$  is a **suffix** of  $S$  with length  $k$  if  $S_k = S[n - k + 1 \cdots n]$ .

## Example:



Prefixes of P:       $P_0 = \varepsilon, P_1 = a, P_2 = ab, P_3 = abb, P_4 = abba$

Suffixes of P:       $S_0 = \varepsilon, S_1 = a, S_2 = ba, S_3 = bba, S_4 = abba$

# String Matching – KMP Algorithm

## Example:

|   |   |   |   |   |   |   |   |   |   |    |
|---|---|---|---|---|---|---|---|---|---|----|
| T | a | b | a | b | b | a | b | b | a | b  |
|   | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 |

|   |   |   |   |   |
|---|---|---|---|---|
| P | a | b | b | a |
|   | 1 | 2 | 3 | 4 |

|       |   |   |   |   |
|-------|---|---|---|---|
| $\pi$ | 0 | 0 | 0 | 1 |
|       | 1 | 2 | 3 | 4 |

$s = \cancel{0}, \cancel{1}, \textcircled{2}, \cancel{3}, \cancel{4}, \textcircled{5}, \cancel{6} = 10 - 4$

The prefix function value is:

$$\pi(q) = k$$

If  $k$  is the length of the longest **prefix**  $P_k$  of  $P_q$  which is a **proper suffix** of  $P_q$ .

# String Matching – KMP Algorithm

Example:

P

a

b

b

a

1

2

3

4

T

a

b

a

b

b

a

b

b

a

b

1

2

3

4

5

6

7

8

9

10

|       |   |   |   |   |
|-------|---|---|---|---|
| $\pi$ | 0 | 0 | 0 | 1 |
|       | 1 | 2 | 3 | 4 |

$s = 0, 1, 2, 3, 4, 5, 6 = 10 - 4$

The prefix function value is:

$$\pi(q) = k$$

If  $k$  is the length of the longest **prefix**  $P_k$  of  $P_q$  which is a **proper suffix** of  $P_q$ .

# String Matching – KMP Algorithm

Example:

P

a

b

b

a

1

2

3

4

T

a

b

a

b

b

a

b

b

a

b

1

2

3

4

5

6

7

8

9

10

|       |   |   |   |   |
|-------|---|---|---|---|
| $\pi$ | 0 | 0 | 0 | 1 |
|       | 1 | 2 | 3 | 4 |

$s = 0, 1, 2, 3, 4, 5, 6 = 10 - 4$

The prefix function value is:

$$\pi(q) = k$$

If  $k$  is the length of the longest **prefix**  $P_k$  of  $P_q$  which is a **proper suffix** of  $P_q$ .

# String Matching – KMP Algorithm

Example:

|   |   |   |   |   |   |   |   |   |   |    |
|---|---|---|---|---|---|---|---|---|---|----|
| P | a | b | b | a |   |   |   |   |   |    |
|   | 1 | 2 | 3 | 4 |   |   |   |   |   |    |
| T | a | b | a | b | b | a | b | b | a | b  |
|   | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 |

|       |   |   |   |   |
|-------|---|---|---|---|
| $\pi$ | 0 | 0 | 0 | 1 |
|       | 1 | 2 | 3 | 4 |

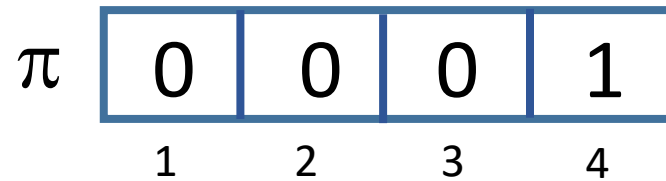
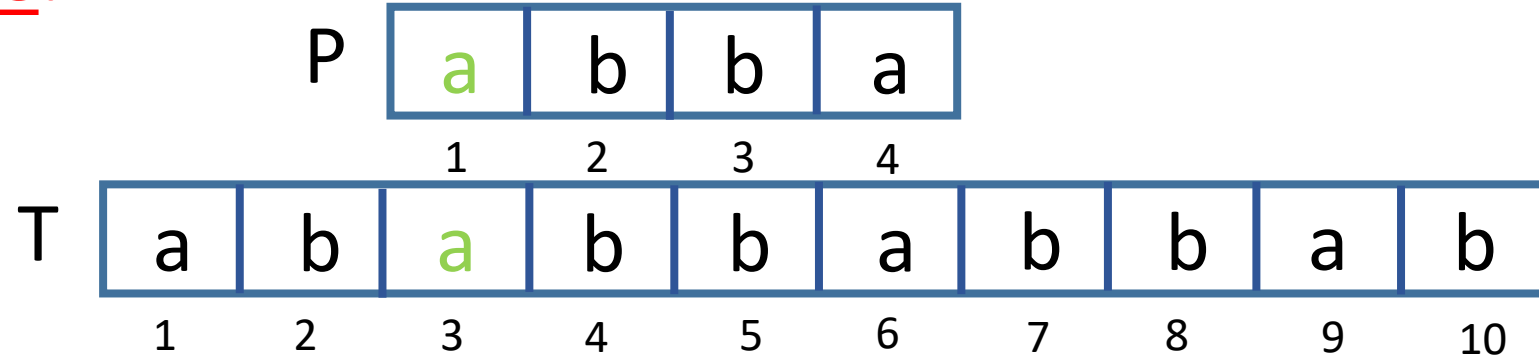
The prefix function value is:

$$\pi(2) = 0$$

$$s = \cancel{0}, 1, 2, 3, 4, 5, 6 = 10 - 4$$

# String Matching – KMP Algorithm

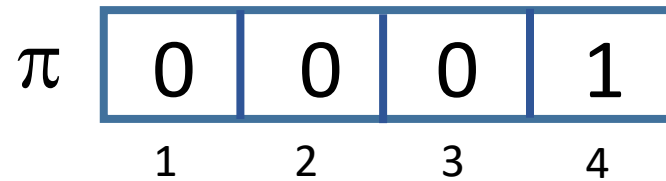
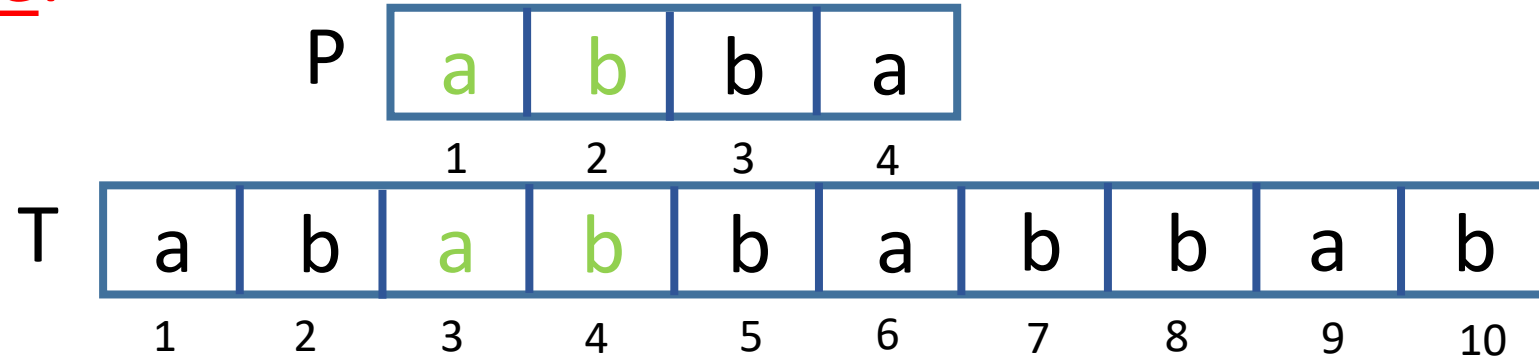
Example:



$$s = \cancel{0}, \cancel{1}, 2, 3, 4, 5, 6 = 10 - 4$$

# String Matching – KMP Algorithm

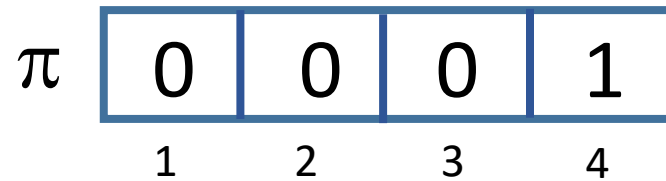
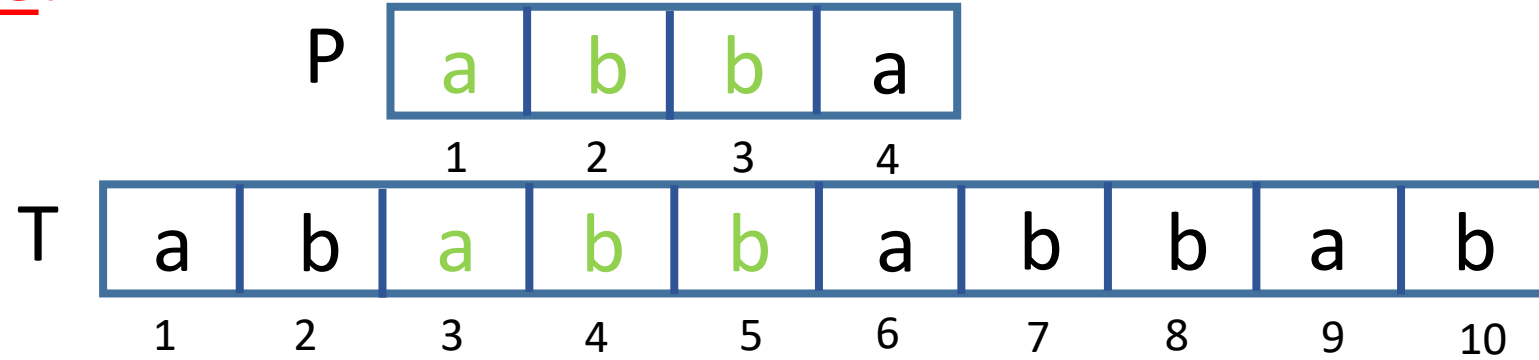
Example:



$$s = \cancel{0}, \cancel{1}, 2, 3, 4, 5, 6 = 10 - 4$$

# String Matching – KMP Algorithm

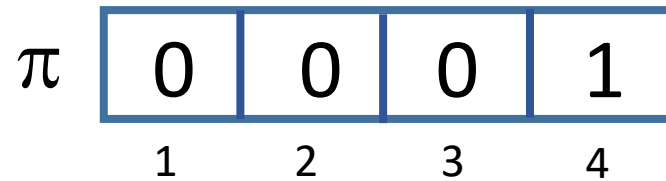
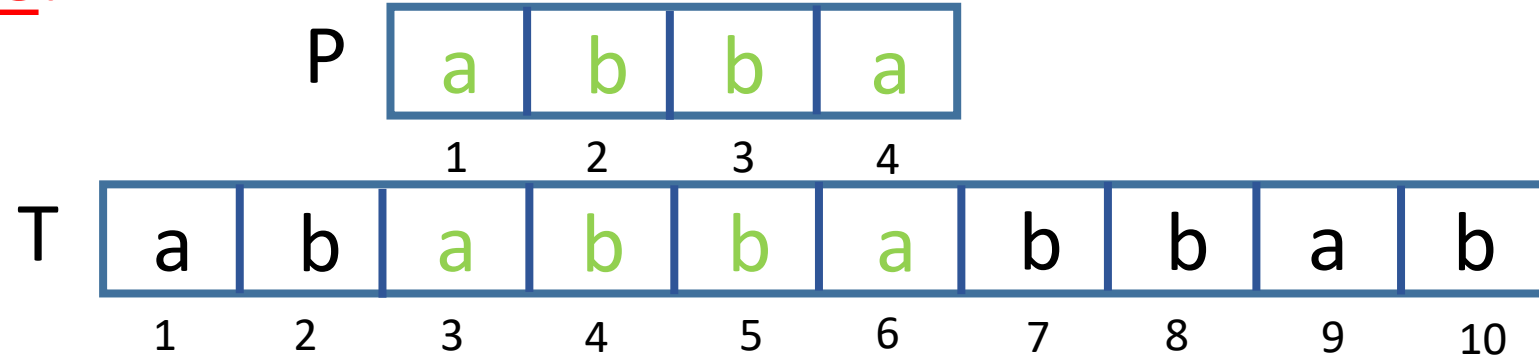
Example:



$$s = \cancel{0}, \cancel{1}, 2, 3, 4, 5, 6 = 10 - 4$$

# String Matching – KMP Algorithm

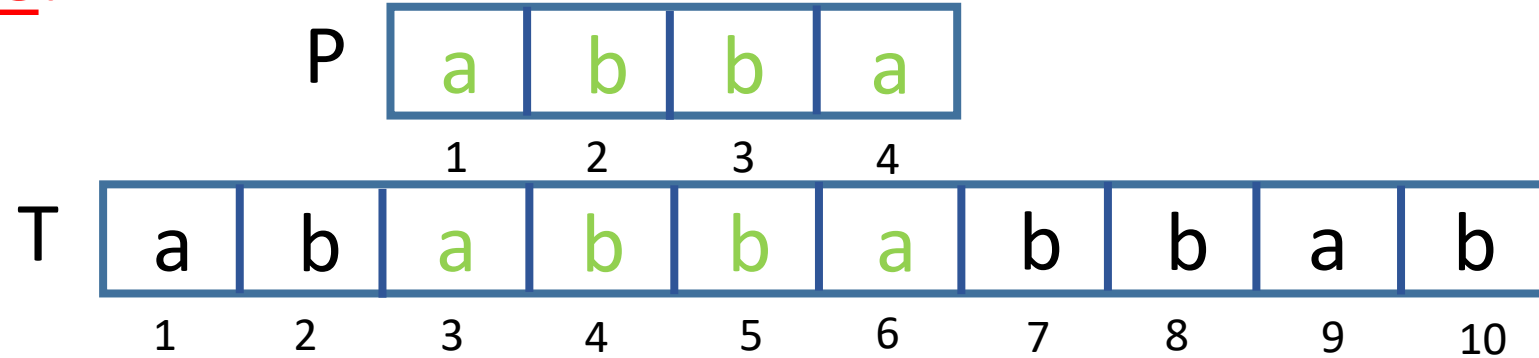
Example:



$$s = \cancel{0}, \cancel{1}, \textcircled{2}, 3, 4, 5, 6 = 10 - 4$$

# String Matching – KMP Algorithm

Example:



The prefix function value is:

$$\pi(4) = 1$$

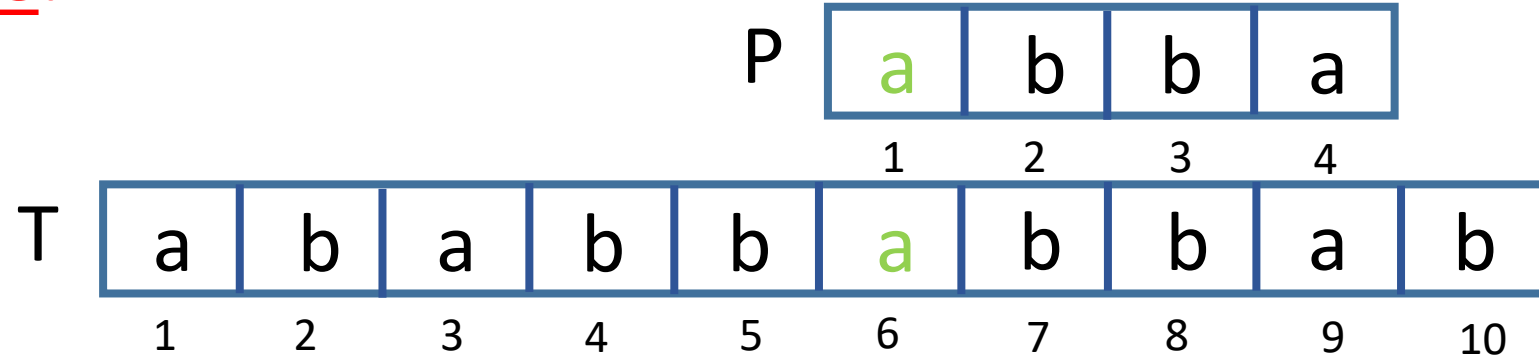
$\pi$

|   |   |   |   |
|---|---|---|---|
| 0 | 0 | 0 | 1 |
| 1 | 2 | 3 | 4 |

$s = \cancel{0}, \cancel{1}, \textcircled{2}, 3, 4, 5, 6 = 10 - 4$

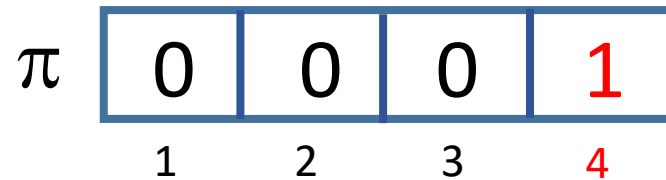
# String Matching – KMP Algorithm

Example:



The prefix function value is:

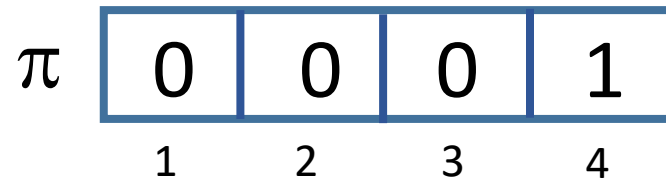
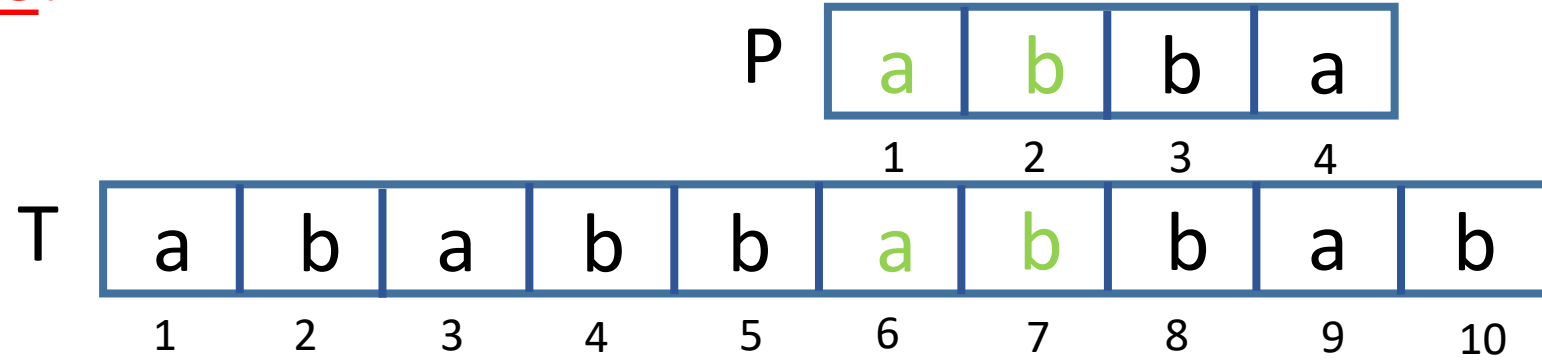
$$\pi(4) = 1$$



$s = 0, 1, 2, 3, 4, 5, 6 = 10 - 4$

# String Matching – KMP Algorithm

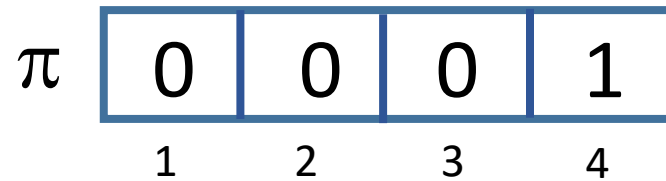
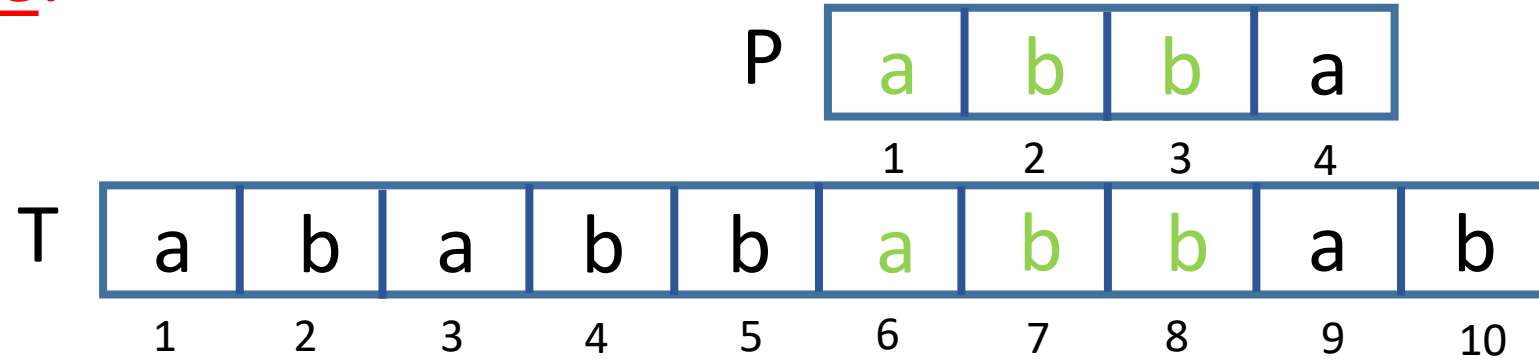
Example:



$$s = \cancel{0}, \cancel{1}, \textcircled{2}, \cancel{3}, \cancel{4}, 5, 6 = 10 - 4$$

# String Matching – KMP Algorithm

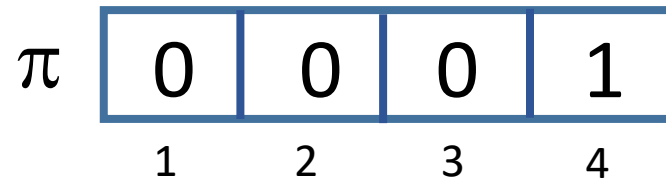
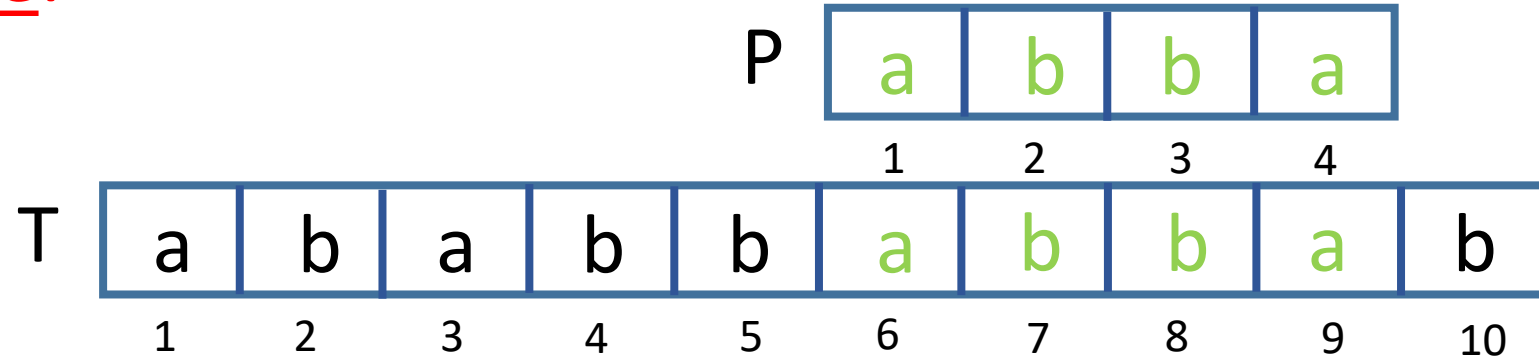
Example:



$s = 0, 1, 2, 3, 4, 5, 6 = 10 - 4$

# String Matching – KMP Algorithm

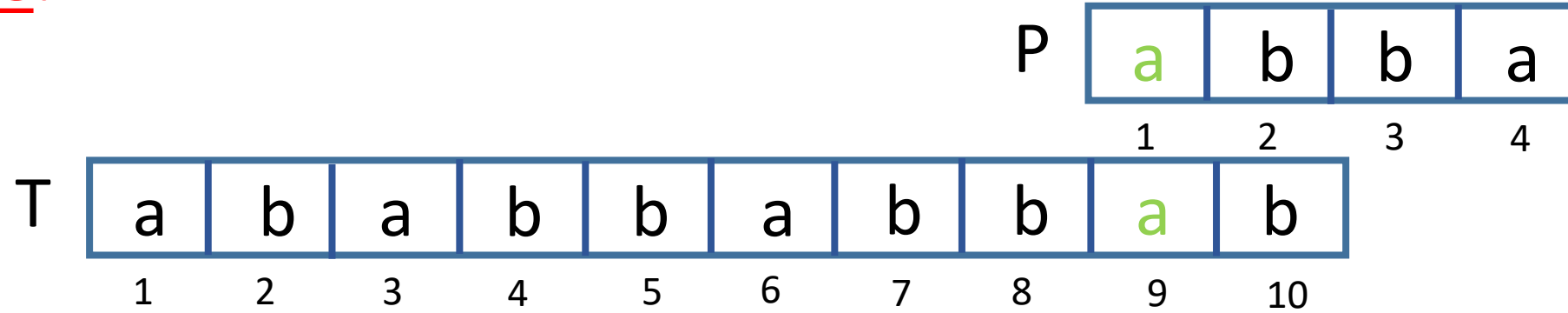
Example:



$s = \cancel{0}, \cancel{1}, \textcircled{2}, \cancel{3}, \cancel{4}, \textcircled{5}, 6 = 10 - 4$

# String Matching – KMP Algorithm

Example:



The prefix function value is:

$$\pi(4) = 1$$

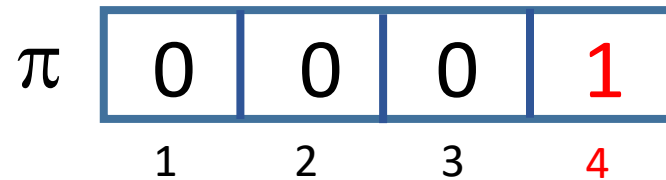


Diagram showing the calculation of the next starting index  $s$  for the next match. The sequence is  $s = 0, 1, 2, 3, 4, 5, 6$ . The values 0, 1, 3, 4, and 6 are crossed out with red slashes. The values 2 and 5 are circled in green. The final result is  $s = 10 - 4 = 6$ .

# String Matching – KMP Algorithm

## Example:

|   |   |   |   |   |   |   |   |   |   |    |
|---|---|---|---|---|---|---|---|---|---|----|
| T | a | b | a | b | b | a | b | b | a | b  |
|   | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 |

|   |   |   |   |   |
|---|---|---|---|---|
| P | b | b | a | b |
|   | 1 | 2 | 3 | 4 |

|       |   |   |   |   |
|-------|---|---|---|---|
| $\pi$ | 0 | 1 | 0 | 1 |
|       | 1 | 2 | 3 | 4 |

$s =$ ~~0~~,~~1~~,~~2~~,3,~~4~~,~~5~~,6 $=10-4$

The prefix function value is:

$$\pi(q) = k$$

If  $k$  is the length of the longest **prefix**  $P_k$  of  $P_q$  which is a **proper suffix** of  $P_q$ .

# String Matching – KMP Algorithm

## Correctness

Claim: (Correctness of Prefix Function Computation)

Let  $P$  be a pattern of length  $m$  given in an array  $P[1 \cdots m]$ .  
Then, the function  $\text{COMPUTE\_PREFIX\_FUNCTION}(P)$  returns the array of the function  $\pi$  with correct values.

Proof:

We show that for each value of  $q$ ,  $1 \leq q \leq m$ , if  $\pi(q) = k$ , then  $k$  is the length of the longest prefix of  $P_q$ , which is a proper suffix of  $P_q$ .

# String Matching – KMP Algorithm

## Correctness

Proof: (cont.)

By induction on  $q$ .

Base case:  $q=1$ .

Trivially, the only possible length for a prefix to be a proper (shorter) suffix of a prefix of length 1 is 0, so line 2 correctly assigns  $\pi(1) = 0$ . This value never changes afterwards.

Induction step: Assume that the function correctly computes  $\pi(q')$  for every  $q'$ ,  $1 \leq q' < q$ . We will show that it also correctly computes  $\pi(q)$ .

# String Matching – KMP Algorithm

## Correctness

Proof: (cont.)

From the former iteration, we know that  $\pi(q - 1) = k$ , where  $k \geq 0$ . Thus, there are two possible cases:

- 1) If  $P[k+1] = P[q]$ : In this case, by induction hypothesis we have that  $k$  is the length of the longest prefix of  $P_{q-1}$ , which is a proper suffix of  $P_{q-1}$ . Since  $P[k+1] = P[q]$ , this prefix can be extended by one character so that  $k+1$  is the length of the longest prefix of  $P_q$ , which is a proper suffix of  $P_q$ . Therefore,  $\pi(q) = k+1$ , as lines 8-9 correctly assign.

# String Matching – KMP Algorithm

## Correctness

Proof: (cont.)

2) If  $P[k+1] \neq P[q]$ : In this case, by induction hypothesis we have that  $k$  is the length of the longest prefix of  $P_{q-1}$ , which is a proper suffix of  $P_{q-1}$ . Since  $P[k+1] \neq P[q]$ , this prefix **cannot** be extended to a prefix of  $P_q$ , which is a proper suffix of  $P_q$ . Therefore,  $\pi(q) < k+1$ .

Note that any prefix of  $P_q$  which is a proper suffix of  $P_q$  with length greater than 0, has the property that its one-character truncated prefix is a prefix of  $P_{q-1}$  which is also a suffix of  $P_{q-1}$ .

# String Matching – KMP Algorithm

## Correctness

Proof: (cont.)

2) If  $P[k+1] \neq P[q]$ :

The while loop in lines 5-6 tries to extend the longest such prefix, which by induction hypothesis was computed correctly and assigned to  $\pi(k)$  (since  $k < q-1$ ).

Since the while loop ends with the longest such prefix that was extended or with 0, line 9 correctly assigns the value of  $\pi(q)$  in either of these cases.

# String Matching – KMP Algorithm

## Correctness

### Theorem: (Correctness of KMP)

Let  $T$  be a text of length  $n$  given in an array  $T[1 \cdots n]$ , and let  $P$  be a pattern of length  $m$  given in an array  $P[1 \cdots m]$ ,  $n > m$ .  
Then, KMP algorithm reports all and only the valid shifts of  $P$  in  $T$ .

### Proof:

We first show that any reported valid shift is indeed valid.  
We then show that any shift that is not reported is indeed invalid.

# String Matching – KMP Algorithm

## Correctness

Proof: (Correctness of KMP cont.)

Note that a shift is reported as valid shift only in line 11 of the algorithm. Prior to the report, an appearance of each of P's subsequent characters is detected, so the identified shift is indeed valid.

Now, any shift that is not reported was either directly rejected by revealing a mismatch in line 6 or 8, which trivially means it is indeed an invalid shift, or implicitly rejected by the prefix function in line 7, where then it must be an invalid shift by the correctness of the prefix function computation.

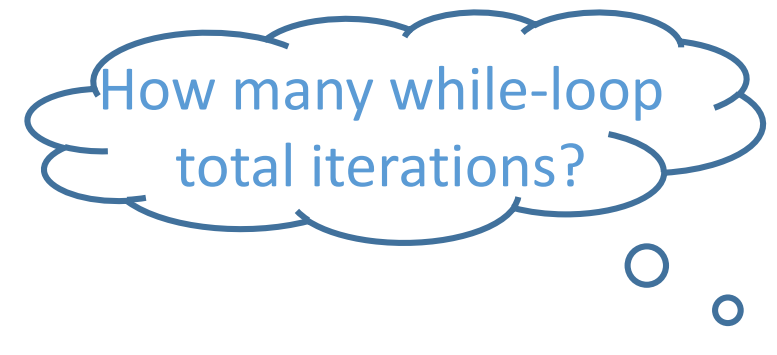
# String Matching – KMP Algorithm Complexity

Claim: (Complexity of Prefix Function Computation)

Let  $P$  be a pattern of length  $m$  given in an array  $P[1 \cdots m]$ .  
Then, the cost of  $\text{COMPUTE\_PREFIX\_FUNCTION}(P)$  function is  $\Theta(m)$ .

Proof:

We show that using the *accounting method* **amortized analysis**.



# String Matching – KMP Algorithm Complexity

Proof: (cont.)

We show that using the *accounting method* **amortized analysis**.

How many while-loop  
total iterations?

k is never negative!  
No overdraft...



Not more than the  
value of k !

If k is 0 in line 5-6,  
while-loop ends!

# String Matching – KMP Algorithm Complexity

Proof: (cont.)

We show that using the *accounting method* **amortized analysis**.

How  $k$  grows?

then  $q$  is  
incremented ...

So  $k$  increments are  
bounded by  $m$  ...



Each iteration  $k$  can be  
incremented by 1 !

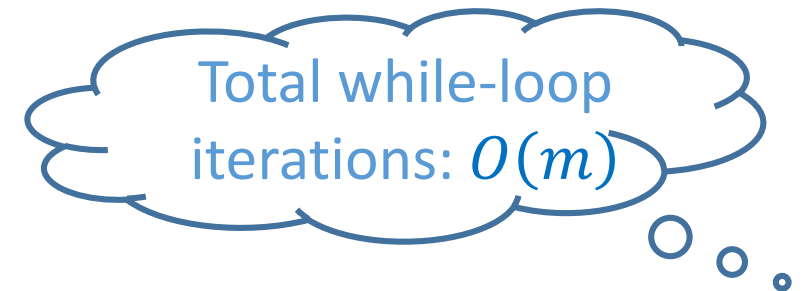
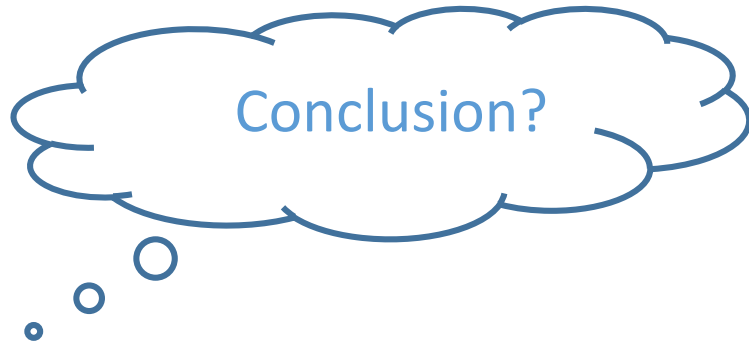
But  $q$  is bounded by  $m$  !

So  $k$  decrements are  
bounded by  $m$  ...

# String Matching – KMP Algorithm Complexity

Proof: (cont.)

We show that using the *accounting method* **amortized analysis**.



# String Matching – KMP Algorithm Complexity

| Command Number | Cost            |
|----------------|-----------------|
| 1-2            | $\Theta(n + m)$ |
| 3              | $\Theta(m)$     |
| 4              | $\Theta(1)$     |
| 5x{6-7}        | $O(n)$          |
| 5x{8-12}       | $\Theta(n)$     |
| Total          | $\Theta(n + m)$ |

